Summarize

Chapter 11. Work Queue Systems

The simplest form of batch processing is a *work queue*. In a work queue system, there is a batch of work to be performed. Each piece of work is wholly independent of the other and can be processed without any interactions. Generally, the goals of the work queue system are to ensure that each piece of work is processed within a certain amount of time. Workers are scaled up or scaled down to ensure that the work can be handled. An illustration of a generic work queue is shown in Figure 11-1.

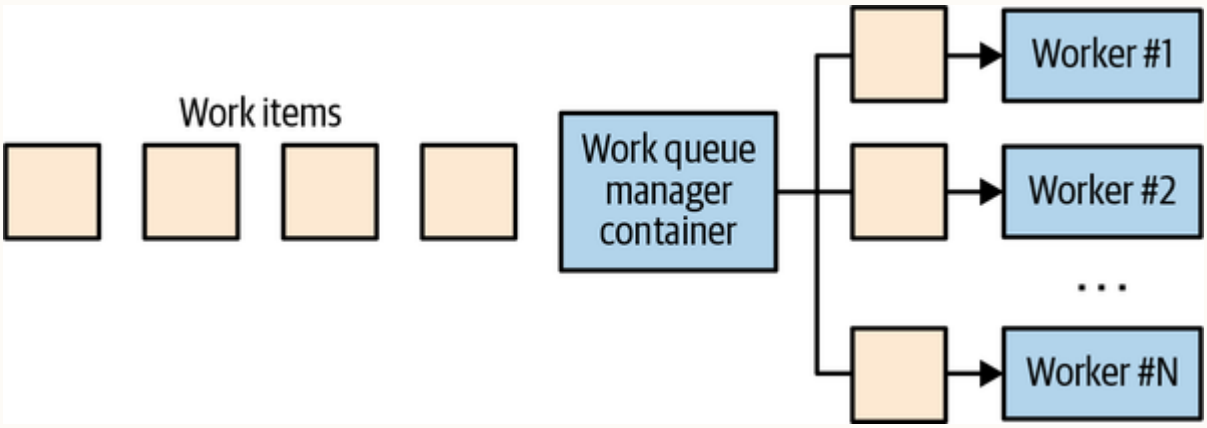


Figure 11-1. A generic work queue

A Generic Work Queue System

The work queue is an ideal way to demonstrate the power of distributed system patterns. Most of the logic in the work queue is wholly independent of the actual work being done, and in many cases the delivery of the work can be performed in an independent manner as well. To illustrate this point, consider the work queue illustrated in Figure 11-1. If we take a look at it again and identify functionality that can be provided by a shared set of *library containers*, it becomes apparent that most of the implementation of a containerized work queue can be shared across a wide variety of users, as shown in Figure 11-2.

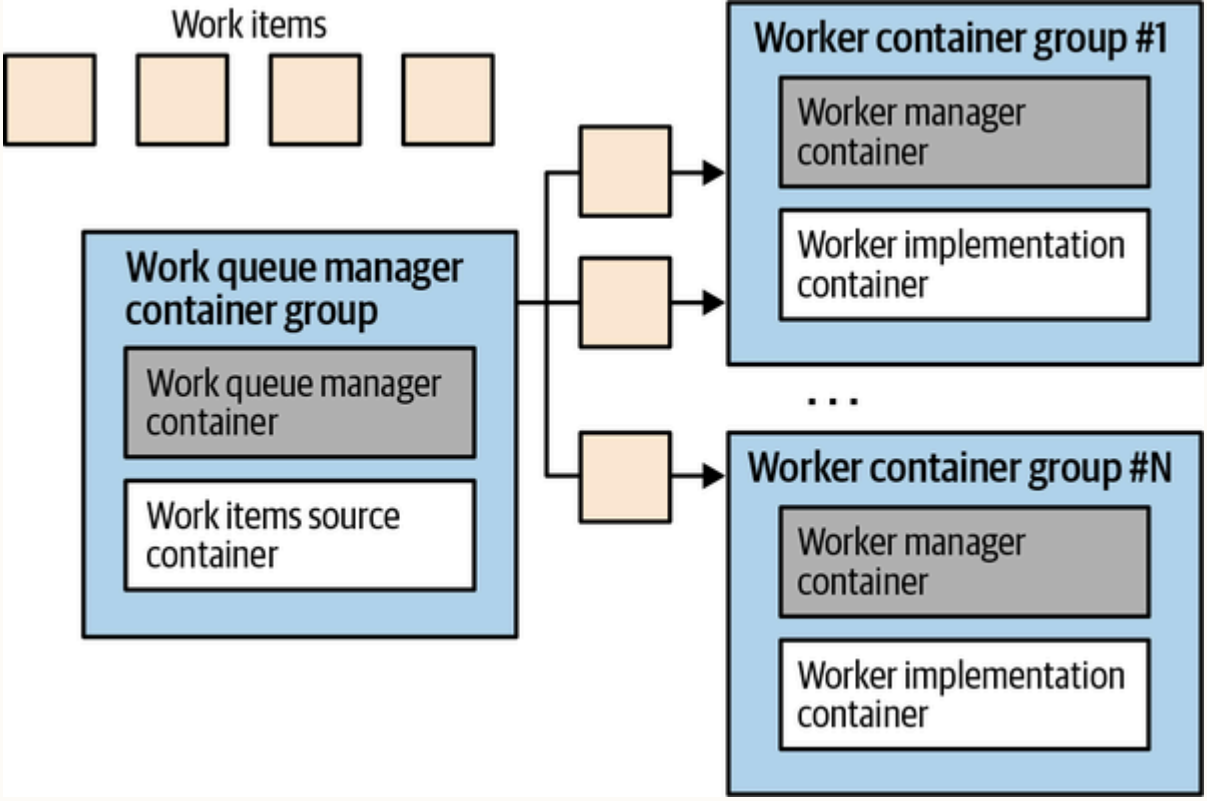


Figure 11-2. The same work queue as shown in Figure 11-1, but this time using reusable containers; the reusable system containers are shown in white, while the user-supplied container is shown in the darker shaded boxes

Building a reusable container-based work queue requires the definition of interfaces between the generic library containers and the user-defined application logic. In the containerized work queue, there are two interfaces: the source container interface, which provides a stream of work items that need processing, and the worker container interface, which knows how to actually process a work item.

The Source Container Interface

To operate, every work queue needs a collection of work items that need processing. There are many different sources of work items for the work queue, depending on the specific application of the work queue. However, once the set of items has been obtained, the actual operation of the work queue is quite generic. Consequently, we can separate the application-specific queue source logic from the generic queue processing logic. Given the previously defined patterns of container groups, this can be seen as an example of the ambassador pattern defined previously. The generic work queue container is the primary application container, and the application-specific source container is the ambassador that proxies the generic work queue's requests out to the concrete definition of the work queue out in the real world. This container group is illustrated in Figure 11-3.

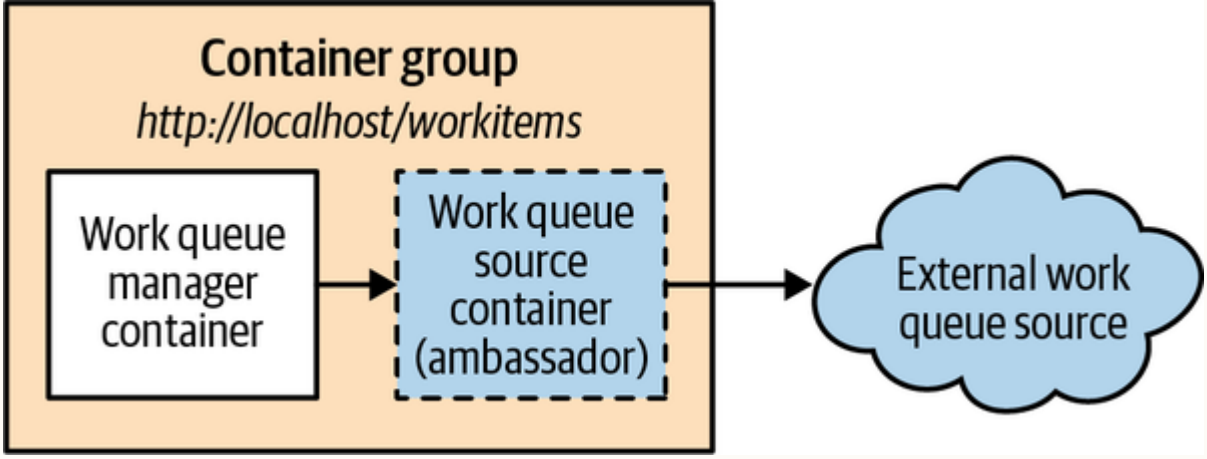


Figure 11-3. The work queue container group

Interestingly, while the ambassador container is clearly application-specific, there is also a variety of generic implementations of the work queue source API. For example, a source might be a list of pictures stored in a cloud storage API, a collection of files stored on network storage, or a queue in a pub/sub system like Kafka or Redis. In these cases, though the user chooses the particular work queue ambassador that fits their scenario, they should be reusing a single generic "library" implementation of the container itself. This minimizes work and maximizes code reuse.

Work Queue API

Given this coordination between the generic work-queue manager and the application-specific ambassador, we need a formal definition of the interface between the two containers. Though there are a variety of different protocols, an HTTP RESTful API is both the easiest to implement as well as the de facto standard for such an interface. The coordinating work queue expects the ambassador to implement the following URLs:

- GET *http://localhost/api/v1/items*
- GET *http://localhost/api/v1/items/<item-name>*

NOTE

You might wonder why we include a *v1* in the API definition. Will there ever be a *v2* of this interface? It may not seem logical, but it costs very little to version your API when you initially define it. Refactoring versioning onto an API without it, on the other hand, is very expensive. Consequently, it is a best practice to always add versions to your APIs even if you're not sure they will ever change. Better safe than sorry.

The */items/* URL returns a complete list of all items:

```
{
  kind: ItemList,
  apiVersion: v1,
  items: [
    "item-1",
    "item-2",
    ....
  ]
}
```

The */items/<item-name>* URL provides the details for a specific item:

```
{
 kind: Item,
 apiVersion: v1,
 data: {
    "some": "json",
  "object": "here",
 }
}
```

Importantly, you will notice that this API doesn't have any affordances for recording that a work item has been processed. We could have designed a more complicated API and then pushed more implementation into the ambassador container, but remember, the goal of this effort is to place as much of the generic implementation inside of the generic work queue manager as possible. To that end, the work queue manager itself is responsible for tracking which items have been processed and which items remain to be processed.

The item details are obtained from this API, and the `item.data` field is passed along to the worker interface for processing.

The Worker Container Interface

Once a particular work item has been obtained by the work queue manager, it needs to be processed by a worker. This is the second container interface in our generic work queue. This container and interface are slightly different than the previous work queue source interface for a few reasons. The first is that it is a one-off API: a single call is made to begin the work, and no other API calls are made throughout the life of the worker container. Secondly, the worker container is not inside a container group with the

work queue manager. Instead, it is launched via a container orchestration API and scheduled to its own container group. This means that the work queue manager has to make a remote call to the worker container in order to start work. It also means that we may need to be more careful about security to prevent a malicious user in our cluster from injecting extra work into the system.

With the work queue source API, we used a simple HTTP-based API for sending items back to the work queue manager. This was because we needed to make repeated calls to the API, and security wasn't a concern since everything was running on localhost. With the worker container, we only need to make a single call, and we want to ensure that other users in the system can't accidentally or maliciously add work to our workers. Consequently, for the worker container, we will use a file-based API. Namely, when the worker container is created, it will receive an environment variable named `WORK_ITEM_FILE`; this will point to a file in the container's local filesystem, where the `data` field from a work item has been written to a file. Concretely, as you will see below, this API can be implemented via a Kubernetes `ConfigMap` object that can be mounted into a container group as a file, as illustrated in Figure 11-4.

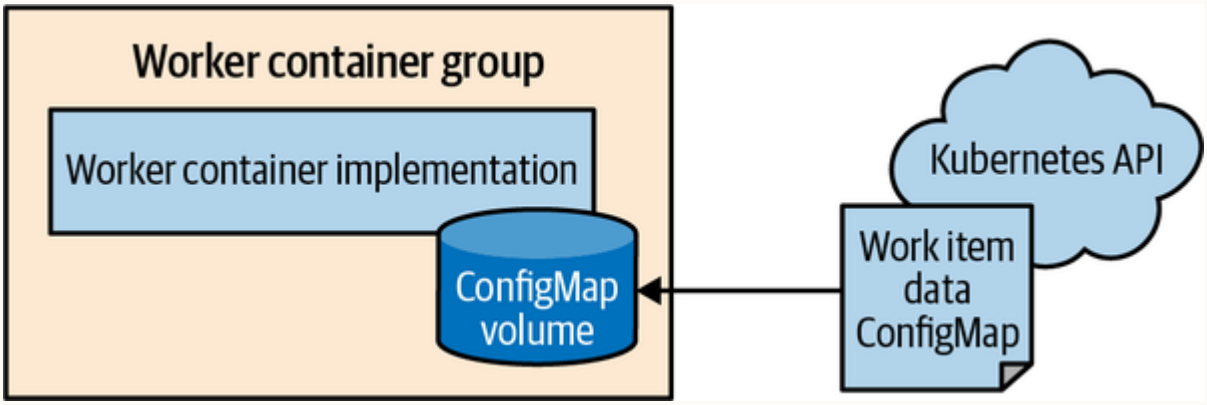


Figure 11-4. The work queue worker API

This file-based API pattern is also easier for the container to implement. Often a work queue worker is simply a shell script across a few command-line tools. In that context, spinning up a web server to manage the work to perform is an unnecessary complexity. As was true with the work queue source implementation, most of the worker containers will be special-purpose container images built for specific work queue applications, but there are also generic workers that can be applied to multiple different work queue applications.

Consider the example of a work queue worker that downloads a file from cloud storage and runs a shell script with that file as input, and then copies its output back up to cloud storage. Such a container can be mostly generic but then have the specific script to execute supplied to it as a runtime parameter. In this way, most of the work of file handling

can be shared by multiple users/work queues and only the specifics of file processing need to be supplied by the end user.

The Shared Work Queue Infrastructure

Given implementations of the two container interfaces described previously, what is left to implement our reusable work queue implementation? The basic algorithm for the work queue is fairly straightforward:

1. Load the available work by calling into source container interface.
2. Consult with work queue state to determine which work items have been processed or are being processed currently.
3. For these items, spawn jobs that use the worker container interface to process the work item.
4. When one of these worker containers finishes successfully, record that the work item has been completed.

While this algorithm is simple to express in words, it is somewhat more complicated to implement in reality. Fortunately for us, the Kubernetes container orchestrator contains a number of features that make it significantly easier to implement. Namely, Kubernetes contains a `Job` object that allows for the reliable execution of the work queue. The `Job` can be configured to either run the worker container once or to run it until it completes successfully. If the worker container is set to run to completion, then even if a machine in the cluster fails, the job will eventually be run to success. This dramatically simplifies the task of building a work queue because the orchestrator takes on responsibility for the reliable operation of each work item.

Additionally, Kubernetes has annotations for each `Job` object that enable us to mark each job with the work item it is processing. This enables us to understand which items are being processed as well as those that have completed in either failure or success.

Put together, this means that we can implement a work queue on top of the Kubernetes orchestration layer without using any storage of our own.

Thus, the expanded operation of our work queue container looks like this:

```
Repeat forever
   Get the list of work items from the work source container interface.
   Get the list of all jobs that have been created to service this work
      queue.
   Difference these lists to find the set of work items that haven't
```

```
    been processed.
  For these unprocessed items, create new Job objects that spawn the
    appropriate worker container.
```

Here is a simple Python script that implements this work queue:

```
import requests
import json
from kubernetes import client, config
import time

namespace = "default"

def make_container(item, obj):
  container = client.V1Container()
  container.image = "my/worker-image"
  container.name = "worker"
  return container

def make_job(item):
  response = requests.get("http://localhost:8000/items/{}".format(item))
  obj = json.loads(response.text)
  job = client.V1Job()
  job.metadata = client.V1ObjectMeta()
  job.metadata.name = item
  job.spec = client.V1JobSpec()
  job.spec.template = client.V1PodTemplate()
  job.spec.template.spec = client.V1PodTemplateSpec()
  job.spec.template.spec.restart_policy = "Never"
  job.spec.template.spec.containers = [
    make_container(item, obj)
  ]
  return job

def update_queue(batch):
  response = requests.get("http://localhost:8000/items")

  obj = json.loads(response.text)
  items = obj['items']

  ret = batch.list_namespaced_job(namespace, watch=False)

  for item in items:
    found = False
    for i in ret.items:
```

```
            if i.metadata.name == item:
                found = True
        if not found:
            # This function creates the job object, omitted for
            # brevity
            job = make_job(item)
            batch.create_namespaced_job(namespace, job)

config.load_kube_config()
batch = client.BatchV1Api()

while True:
    update_queue(batch)
    time.sleep(10)
```

Hands On: Implementing a Video Thumbnailer

To provide a concrete example of how we might use a work queue, consider the task of generating thumbnails for videos. These thumbnails help users determine which videos they want to watch.

To implement this video thumbnailer, we need two different user containers. The first is the work item source container. The simplest way for this to work is for the work items to appear on a shared disk, such as a Network File System (NFS) share. The work item source simply lists the files in this directory and returns them to the caller. Here's a simple `node` program that does this:

```
const http = require('http');
const fs = require('fs');

const port = 8080;
const path = process.env.MEDIA_PATH;

const requestHandler = (request, response) => {
        console.log(request.url);
        fs.readdir(path + '/*.mp4', (err, items) => {
                var msg = {
                        'kind': 'ItemList',
                        'apiVersion': 'v1',
                        'items': []
                };
                if (!items) {
```

```
                return msg;
            }
            for (var i = 0; i < items.length; i++) {
                msg.items.push(items[i]);
            }
            response.end(JSON.stringify(msg));
        });
}

const server = http.createServer(requestHandler);

server.listen(port, (err) => {
        if (err) {
            return console.log('Error starting server', err);
        }

        console.log(`server is active on ${port}`)
});
```

This source of defines the queue of movies to thumbnail. We use the `ffmpeg` utility to actually perform the thumbnailing work.

You can create a container that uses the following as its command line:

```
ffmpeg -i ${INPUT_FILE} -frames:v 100 thumb.png
```

This command will take one frame every 100 frames (that's the `-frames:v 100` flag) and output it as a PNG file (e.g., `thumb1.png`, `thumb2.png`, etc.).

You can create the `ffmpeg` container using a base image of your choice and installing the `ffmpeg` executable in that container. There is also an open source project with Dockerfiles for many popular operating systems on GitHub. You can use those scripts as a starting point for your container image.

By defining a simple source container as well as an even simpler worker container, we can clearly see the power and utility of a generic container-based queuing system. It dramatically reduces the time/distance between an idea for implementing a work queue and the corresponding concrete implementation.

Dynamic Scaling of the Workers

The previously described work queue is great for processing work items as quickly as they arrive in the work queue, but this can lead to bursty resource loads being placed onto a container orchestrator cluster. This is good if you have a lot of different workloads that will burst at different times and thus keep your infrastructure evenly utilized. But if you don't have a sufficient number of different workloads, this feast or famine approach to scaling your work queue might require that you over-provision resources to support the bursts that will lie idle (and cost too much money) while you don't have work to perform.

To address this problem, you can limit the overall number of `Job` objects that your work queue is willing to create. This will naturally serve to limit the number of work items you process in parallel and consequentially limit the maximum amount of resources that you use at a particular time. However, doing this will increase the time to completion (latency) for each work item being completed when under heavy load. If the load is bursty, then this is probably OK because you can use the slack times to catch up with the backlog that developed during a burst of usage. However, if your steady-state usage is too high, your work queue may never be able to catch up, and the time to completion will simply get longer and longer.

When your work queue is faced with this situation, you need to have it dynamically adjust itself to increase the parallelism that it is willing to create (and correspondingly the resources it is willing to use) so that it can keep up with the incoming work. Fortunately, there are mathematical formulas that we can use to determine when we need to dynamically scale up our work queue.

Consider a work queue where a new work item arrives an average of once every minute, and each work item takes an average of 30 seconds to complete. Such a system is capable of keeping up with all of the work it receives. Even if a large batch of work arrives all at once and creates a backlog, on average the work queue processes two work items for every one work item that arrives, and thus it will be able to gradually work through its backlog.

If, instead, a new work item arrives on average once every minute and it takes an average of one minute to process each work item, then the system is perfectly balanced, but it does not handle variance well. It can catch up with bursts—but it will take a while, and it has no slack or capacity to absorb a sustained increase in the rate at which new work items arrive. This is probably not an ideal way to run, as some safety margin for growth

and other sustained increases in work (or unexpected slowdowns in processing) is needed to preserve a stable system.

Finally, consider a system in which a work item arrives every minute and each item takes two minutes to process. In such a system, we are always losing ground. The queue of work will grow without bound, and the latency of any one item in the queue will grow toward infinity (and our users will become very frustrated).

Thus, we can keep track of both of these metrics for our work queue, and the average time between work items over an extended period of time (# work items / 24 hours) will give us the *interarrival time* for new work. We can also keep track of the average *latency*, the time to process any one item once we start working on it, not counting any time spent waiting in the queue. To have a stable work queue, we need to adjust the number of resources so that latency is less than the interarrival time of new items. If we are processing work items in parallel, we also divide the processing time for a work item by the parallelism. For example, if each item takes one minute to process but we process four items in parallel, the effective time to process one item is 15 seconds, and thus we can sustain an interarrival period of 16 or more seconds.

This approach makes it fairly straightforward to build an autoscaler to dynamically size up our work queue. Sizing down the work queue is somewhat trickier, but you can use the same math as well as a heuristic for the amount of spare capacity for the safety margin you want to maintain. For example, you can reduce the parallelism until the processing time for an item is 90% of the interarrival time for new items. While in some cases it is more efficient to build your own scaler, many times it is better to use an open source solution. That is where the Kubernetes Event-Driven Autoscaling (KEDA) project is useful. KEDA provides numerous different event-based scalers, which can be used to scale your application. KEDA supports event sources ranging from files on disk to events in pub/sub systems and many more.

The Multiworker Pattern

One of the themes of this book has been the use of containers for encapsulation and reuse of code. The same holds true for the work queue patterns described in this chapter. In addition to the patterns for reusing containers for driving the work queue itself, you can also reuse multiple different containers to compose a worker implementation. Suppose, for example, that you have three different types of work that you want to perform on a particular work queue item. For example, you might want to detect faces in an image, tag those faces with identities, and then blur the faces in the image. You could

write a single worker to perform this complete set of tasks, but this would be a bespoke solution that would not be reusable the next time you want to identify something else, such as cars, yet still provide the same blurring.

To achieve this kind of code reuse, the *multiworker pattern* is something of a specialization of the adapter pattern described in previous chapters. In this case, the multiworker pattern transforms a collection of different worker containers into a single unified container that implements the worker interface, yet delegates the actual work to a collection of different reusable containers. This process is illustrated in Figure 11-5.

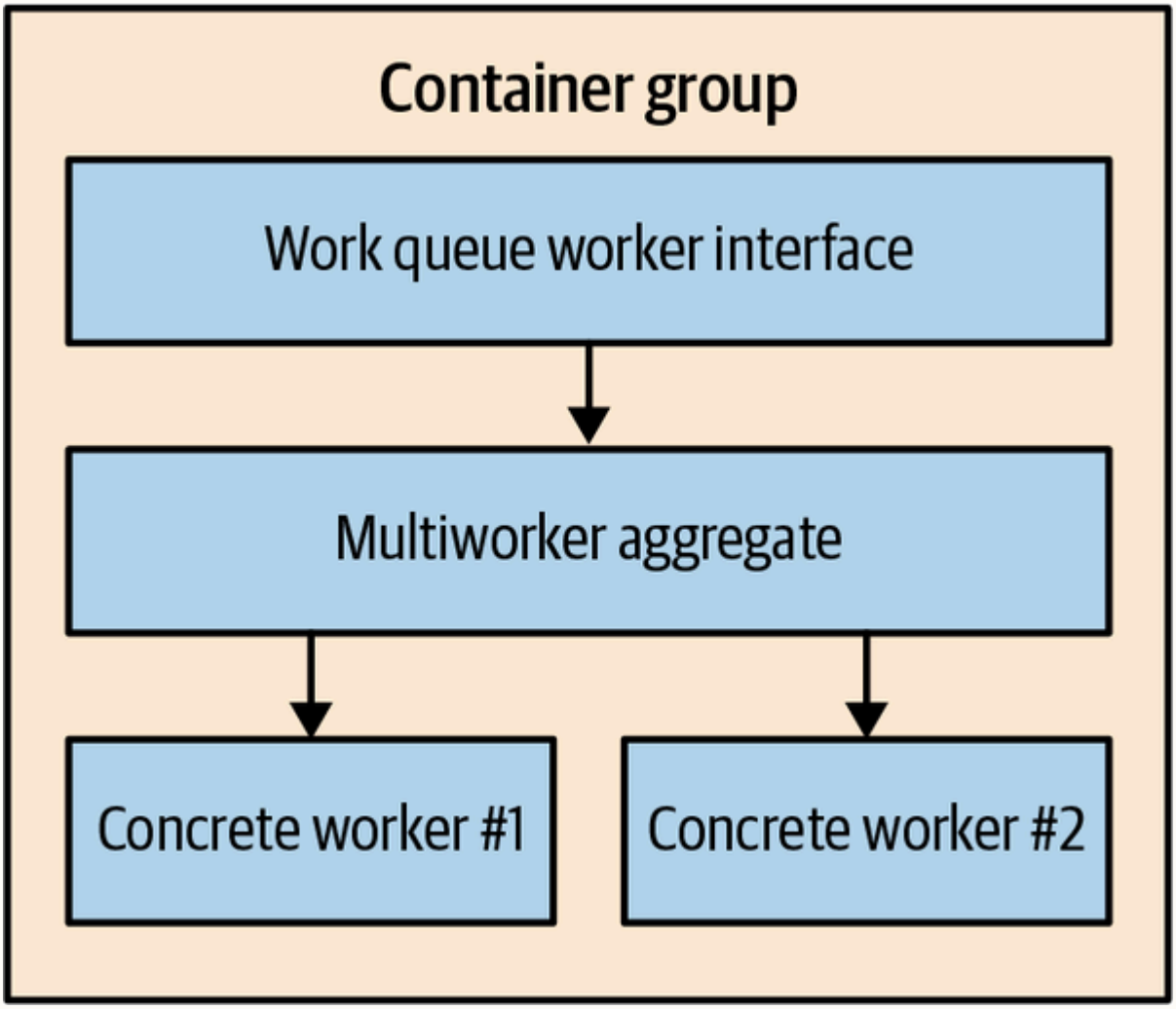


Figure 11-5. The multiworker aggregator pattern as a group of containers

Because of this code reuse, the composition of multiple different worker containers means an increase in the reuse of code and a reduction in effort for people designing batch-oriented distributed systems.

Summary

This chapter begins the discussion of batch systems by introducing *workflows*. A workflow is a sequence of discrete actions which, when put together in a sequence, achieve

some goal like the processing of an order or the machine-learning analysis of a video. This chapter showed how containers can be used to build modular workflows with sidecar containers providing workflow orchestration while application containers supply the specific processing necessary for your workflow. Workflow is a key part of most distributed batch systems. Its generic pattern is easy to adapt and extend to meet the specific needs of your workload.

Previous chapter
IV. Batch Computational Patterns

Next chapter
12. Event-Driven Batch Processing